

ECO 529: Macro-Finance: Tools, Models, and Ideas

Lecture 4: Solving Differential Equations Numerically

Jonathan Payne

3rd November, 2025

“The question of whether a computer can think is no more interesting than the question of whether a submarine can swim.”

–Edsger W. Dijkstra

Goals

- ▶ “Traditional” approaches to solving differential equations (especially finite difference)
- ▶ Introduce neural networks
- ▶ Outline algorithms for training neural networks
- ▶ Discuss different software packages
- ▶ Apply algorithms to solving differential equations
- ▶ Aim: everyone solves a differential equation with deep learning

References on Traditional Techniques for Solving PDEs

- ▶ Finite differences: [LeVeque, 2007], [Presse et al., 1988]
- ▶ Spectral method: [Pozrikidis, 2005], [Trefethen, 2001]
- ▶ Other references:
 - ▶ Differential equation theory: Evans (2001)
 - ▶ Technique comparisons: [Pichler et al., 2013], [Trefethen, 1996]
 - ▶ Introductory texts: [Holmes, 2007], [Iserles, 2008]
 - ▶ Upwind scheme in macroeconomic models: [Candler, 2001]

Suggested References on Deep Learning

- ▶ [Goodfellow et al., 2016]: “Deep Learning”
 - ▶ Typically recommended as a mathematical reference on deep learning
 - ▶ Doesn't provide coding examples
- ▶ [Zhang et al., 2021]: “Dive Into Deep Learning”
 - ▶ Builds python code from “from scratch”, which helps “de-mystify” NNs
 - ▶ Also provides sample code in TensorFlow, PyTorch and Gluon
- ▶ [Géron, 2019]: “Hands-on Machine Learning”. 2nd Edition.
 - ▶ 2nd Edition introduces Neural Networks using the Keras interface to TensorFlow 2 (one of the main deep learning systems)
 - ▶ Doesn't cover mathematical details

Table of Contents

An Test Model of Population Evolution

Finite Difference

Example 1: Steady State KFE With Exponential Entry Rate

Example 2: Neumann Boundary Conditions

Example 4: Initial Value Problem

Artificial Neural Networks

Introduction to Neural Nets

Solving Differential Equations With Neural Networks

Numerical Packages

Numerical Examples

Optional Appendix: Spectral Method

General Principles

Example 1: Linear ODE

Example 4: (Revisited)

Population model

- ▶ Suppose that agents have an individual state evolving according to $dX_t^i = \sqrt{2}\sigma dW_t^i$.
- ▶ Agents exit when they reach $x \in (x_L, x_H)$.
- ▶ New agents arrive at x at rate $\lambda_b(x) = f(x)$.

What is the KFE for this population?

KFE

- Recall: the generic KFE (for Brownian rather than Poisson shocks) is:

$$\partial_t g_t(x) = \lambda_b(x) - \lambda_k(x)g_t(x) - \partial_x[\mu_t(x)g_t(x)] + \frac{1}{2}\partial_{xx}[\sigma_t^2(x)g_t(x)]$$

- So for individual evolution $dX_t^i = \sqrt{2}\sigma dW_t^i$ and absorbing boundaries the KFE is:
(In maths, this is called the heat equation or diffusion equation)

$$\partial_t g_t(x) = f(x) + \sigma^2 \partial_{xx} g_t(x), \quad g(x_L) = 0, \quad g(x_H) = 0$$

- We start by solving for the steady state heat equation for $\sigma = 1$ (which is also the 1D Poisson equation). This is a *boundary value problem*.

$$0 = f(x) + \partial_{xx} g_t(x), \quad \text{s.t.} \quad g(x_L) = 0, \quad g(x_H) = 0$$

- We then solve the full, time varying heat equation. This is an *initial value problem*.

$$\partial_t g_t(x) = f(x) + \sigma^2 \partial_{xx} g_t(x), \quad \text{s.t.} \quad \text{boundary conditions} + \text{initial value conditions}$$

Table of Contents

An Test Model of Population Evolution

Finite Difference

Example 1: Steady State KFE With Exponential Entry Rate

Example 2: Neumann Boundary Conditions

Example 4: Initial Value Problem

Artificial Neural Networks

Introduction to Neural Nets

Solving Differential Equations With Neural Networks

Numerical Packages

Numerical Examples

Optional Appendix: Spectral Method

General Principles

Example 1: Linear ODE

Example 4: (Revisited)

Table of Contents

An Test Model of Population Evolution

Finite Difference

Example 1: Steady State KFE With Exponential Entry Rate

Example 2: Neumann Boundary Conditions

Example 4: Initial Value Problem

Artificial Neural Networks

Introduction to Neural Nets

Solving Differential Equations With Neural Networks

Numerical Packages

Numerical Examples

Optional Appendix: Spectral Method

General Principles

Example 1: Linear ODE

Example 4: (Revisited)

Example 1: Steady State KFE With Exponential Entry Rate

- **Q. Problem:** Consider the differential equation (steady state diffusion / 1D Poisson)

$$0 = f(x) + \partial_{xx}g(x), \quad \text{where } x \in (x_L, x_H), \quad f(x) = e^x$$

subject to the boundary conditions (referred to as “Dirichlet” boundary conditions):

$$g(x_L) = g_L = 0, \qquad g(x_H) = g_H = 0$$

- **A. Analytic solution:** Integrating twice (or using Mathematica) gives general solution:

$$g(x) = -e^x + Cx + D$$

Substituting in the boundary condition solving for C and D gives:

$$C = \frac{g_H - g_L + (e^{x_H} - e^{x_L})}{x_H - x_L} \qquad D = g_H + e^{x_H} - Cx_H$$

How can we solve this numerically with finite difference?

Discretize state space

- ▶ Discretize interval $[x_L, x_H]$ into N equally spaced grid points:

$$\{x_1, x_2, \dots, x_{N-1}, x_N\}, \quad x_1 = x_L, \quad x_N = x_H$$

- ▶ Distance between grid points (mesh width) is:

$$h = \frac{x_n - x_1}{N - 1}$$

- ▶ Discretize $g(x)$ on $\{x_1, x_2, \dots, x_{N-1}, x_N\}$. Denote the discretized values of g by:

$$\{G_1, \dots, G_N\} = \{g(x_1), \dots, g(x_N)\}$$

Finite Difference Approximation to $\partial_x g(x)$

- ▶ We only need to approximate $\partial_{xx}g(x)$, but let's consider $\partial_x g(x)$ now anyway
- ▶ **Forward difference** at grid point x_j is: (1st order accurate; error $o(h)$)

$$\hat{\partial}_{F,j}g(x) := \frac{G_{j+1} - G_j}{x_{j+1} - x_j} = \frac{G_{j+1} - G_j}{h}, \quad 1 \leq j \leq N - 1$$

- ▶ **Backward difference** at grid point x_j is: (1st order accurate; error $o(h)$)

$$\hat{\partial}_{B,j}g(x) := \frac{G_j - G_{j-1}}{x_j - x_{j-1}} = \frac{G_j - G_{j-1}}{h}, \quad 2 \leq j \leq N$$

- ▶ See [LeVeque \(2007\)](#), Chp 1 for derivation from Taylor series.

Finite Difference Approximation to $\partial_{xx}g(x)$

- **Centered approximation** at grid point x_j is: (4th order accurate; error $o(h^2)$)

$$\hat{\partial}_{C,j}^2 g(x) = \hat{\partial}_{F,j} \hat{\partial}_{B,j} g(x) = \frac{G_{j+1} - 2G_j + G_{j-1}}{h^2}, \quad 2 \leq j \leq N-1$$

- See [LeVeque \(2007\)](#), Chp 1 for derivation from Taylor series.

Finite Difference Approximation

- Discretized DE becomes a system of algebraic equations:

$$G_1 = g_L, \quad j = 1$$

$$\frac{1}{h^2}(G_{j-1} - 2G_j + G_{j+1}) = -f(x_j), \quad 2 \leq j \leq N-1$$

$$G_N = g_H, \quad j = N$$

- Alternative: Stack equations 1 to N (all equations) into matrix form:

$$\underbrace{\frac{1}{h^2} \begin{bmatrix} h^2 & 0 & 0 & 0 & 0 & \dots & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 & \dots & 0 & 0 \\ & & & \ddots & \ddots & \ddots & & \\ 0 & 0 & \dots & 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & \dots & 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & \dots & 0 & 0 & 0 & 0 & h^2 \end{bmatrix}}_{:=A} \underbrace{\begin{bmatrix} G_1 \\ G_2 \\ G_3 \\ \vdots \\ G_{N-2} \\ G_{N-1} \\ G_N \end{bmatrix}}_{:=G} = \underbrace{\begin{bmatrix} g_L \\ -f(x_2) \\ -f(x_3) \\ \vdots \\ -f(x_{N-2}) \\ -f(x_{N-1}) \\ g_H \end{bmatrix}}_{:=F}$$

- Can solve: $AG = F \Rightarrow G = A^{-1}F$

Alternative Matrix Formation (Without Boundaries)

- Discretized DE becomes a system of algebraic equations:

$$G_1 = g_L, \quad j = 1$$

$$\frac{1}{h^2}(G_{j-1} - 2G_j + G_{j+1}) = -f(x_j), \quad 2 \leq j \leq N-1$$

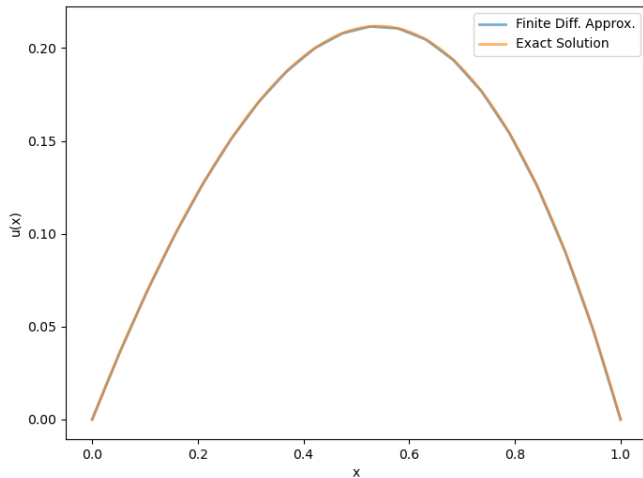
$$G_N = g_H, \quad j = N$$

- Sub in G_1 and G_N and stack equations 2 to $N-1$ into matrix form:

$$\underbrace{\frac{1}{h^2} \begin{bmatrix} -2 & 1 & 0 & 0 & \dots & 0 \\ 1 & -2 & 1 & 0 & \dots & 0 \\ 0 & 1 & -2 & 1 & \dots & 0 \\ & & \ddots & \ddots & \ddots & \\ 0 & \dots & 0 & 1 & -2 & 1 \\ 0 & \dots & 0 & 0 & 1 & -2 \end{bmatrix}}_{:=A} \underbrace{\begin{bmatrix} G_2 \\ G_3 \\ G_4 \\ \vdots \\ G_{N-2} \\ G_{N-1} \end{bmatrix}}_{:=G} = \underbrace{\begin{bmatrix} -f(x_2) - g_L/h^2 \\ -f(x_3) \\ -f(x_4) \\ \vdots \\ -f(x_{N-2}) \\ -f(x_{N-1}) - g_H/h^2 \end{bmatrix}}_{:=F}$$

- Can solve: $AG = F \Rightarrow G = A^{-1}F$

Sample Code and Solution



Julia/FD_Ex01_PoisEqnSolver_DirBC.jl

Table of Contents

An Test Model of Population Evolution

Finite Difference

Example 1: Steady State KFE With Exponential Entry Rate

Example 2: Neumann Boundary Conditions

Example 4: Initial Value Problem

Artificial Neural Networks

Introduction to Neural Nets

Solving Differential Equations With Neural Networks

Numerical Packages

Numerical Examples

Optional Appendix: Spectral Method

General Principles

Example 1: Linear ODE

Example 4: (Revisited)

Example 2: Neumann Boundary Conditions

- ▶ Consider diffusion equation:

$$0 = f(x) + \partial_{xx}g(x), \quad x \in (x_L, x_H)$$

- ▶ Subject to the “mixed Dirichlet and Neumann” boundary conditions:

$$\partial_x g(x_L) = 0$$

$$g(x_H) = 0$$

- ▶ and the forcing term:

$$f(x) = e^x$$

- ▶ Q. What population model would this correspond to?

Analytical Solution

- ▶ Integrate twice and apply boundary conditions.
- ▶ This gives the solution:

$$g(x) = -e^x + Cx + D$$

- ▶ where

$$C = g_L + e^{x_L}$$

$$D = g_H + e^{x_H} - Cx_H$$

Finite Difference Approximation

- Approximate $g_x(x_L)$ using forward difference (because at lower boundary):

$$\frac{G_2 - G_1}{h} = g_L$$

Stack other equations the same way; gives matrix form:

$$\underbrace{\frac{1}{h^2} \begin{bmatrix} -h & h & 0 & 0 & \dots & 0 \\ 1 & -2 & 1 & 0 & \dots & 0 \\ 0 & 1 & -2 & 1 & \dots & 0 \\ 0 & 0 & 1 & -2 & \dots & 0 \\ & & \ddots & \ddots & \ddots & \\ 0 & \dots & 1 & -1 & 1 & 0 \\ 0 & \dots & 0 & 1 & -2 & 1 \\ 0 & \dots & 0 & 0 & 0 & h^2 \end{bmatrix}}_{:=A} \underbrace{\begin{bmatrix} G_1 \\ G_2 \\ G_3 \\ G_4 \\ \vdots \\ G_{N-2} \\ G_{N-1} \\ G_N \end{bmatrix}}_{:=G} = \underbrace{\begin{bmatrix} g_L \\ -f(x_2) \\ -f(x_3) \\ -f(x_4) \\ \vdots \\ -f(x_{N-2}) \\ -f(x_{N-1}) \\ g_H \end{bmatrix}}_{:=F}$$

Can solve: $G = A^{-1}F$

Finite Difference Approximation (Version 2)

- Introduce G_0 , then approximate $\partial_{xx}g(x_L)$ and $\partial_xg(x_L)$ using central difference:

$$\left. \begin{aligned} \frac{1}{h^2}(G_0 - 2G_1 + G_2) &= -f(x_1), & j=0 \\ \frac{1}{2h}(G_2 - G_0) &= g_L, & j=1 \end{aligned} \right\} \Rightarrow \frac{1}{h}(-G_1 + G_2) = g_L - \frac{h}{2}f(x_1)$$

- Stack equations 0 to N into matrix form and solve: $G = A^{-1}F$:

$$\underbrace{\frac{1}{h^2} \begin{bmatrix} -h & h & 0 & 0 & \dots & 0 \\ 1 & -2 & 1 & 0 & \dots & 0 \\ 0 & 1 & -2 & 1 & \dots & 0 \\ 0 & 0 & 1 & -2 & \dots & 0 \\ & & \ddots & \ddots & \ddots & \\ 0 & \dots & 1 & -2 & 1 & 0 \\ 0 & \dots & 0 & 1 & -2 & 1 \\ 0 & \dots & 0 & 0 & 0 & h^2 \end{bmatrix}}_{:=A} \underbrace{\begin{bmatrix} G_1 \\ G_2 \\ G_3 \\ G_4 \\ \vdots \\ G_{N-2} \\ G_{N-1} \\ G_N \end{bmatrix}}_{:=G} = \underbrace{\begin{bmatrix} g_L - \frac{h}{2}f(x_0) \\ -f(x_2) \\ -f(x_3) \\ -f(x_4) \\ \vdots \\ -f(x_{N-2}) \\ -f(x_{N-1}) \\ g_H \end{bmatrix}}_{:=F}$$

Finite Difference Approximation (Version 3)

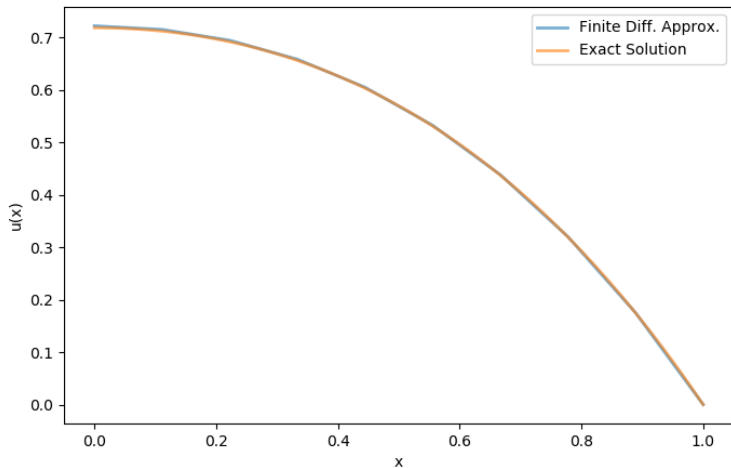
- Use a right one sided approximation accurate to high order $o(h^3)$ (can check this is $o(h^3)$ using Taylor approximation, see Example 1.2 in LeVeque (2007)):

$$(1/h) ((3/2)G_1 - 2G_2 + (1/2)G_3)$$

- Stack equations 0 to N into matrix form and solve for $G = A^{-1}F$:

$$\underbrace{\frac{1}{h^2} \begin{bmatrix} -3h/2 & -2h & h/2 & 0 & \dots & 0 \\ 1 & -2 & 1 & 0 & \dots & 0 \\ 0 & 1 & -2 & 1 & \dots & 0 \\ 0 & 0 & 1 & -2 & \dots & 0 \\ & & \ddots & \ddots & \ddots & \\ 0 & \dots & 1 & -2 & 1 & 0 \\ 0 & \dots & 0 & 1 & -2 & 1 \\ 0 & \dots & 0 & 0 & 0 & h^2 \end{bmatrix}}_{:=A} \underbrace{\begin{bmatrix} G_1 \\ G_2 \\ G_3 \\ G_4 \\ \vdots \\ G_{N-2} \\ G_{N-1} \\ G_N \end{bmatrix}}_{:=G} = \underbrace{\begin{bmatrix} g_L \\ -f(x_2) \\ -f(x_3) \\ -f(x_4) \\ \vdots \\ -f(x_{N-2}) \\ -f(x_{N-1}) \\ g_H \end{bmatrix}}_{:=F}$$

Sample Code and Solution



Julia/FD_Ex02_PoisEqnSolver_FD_NeuBC.jl

Table of Contents

An Test Model of Population Evolution

Finite Difference

Example 1: Steady State KFE With Exponential Entry Rate

Example 2: Neumann Boundary Conditions

Example 4: Initial Value Problem

Artificial Neural Networks

Introduction to Neural Nets

Solving Differential Equations With Neural Networks

Numerical Packages

Numerical Examples

Optional Appendix: Spectral Method

General Principles

Example 1: Linear ODE

Example 4: (Revisited)

Initial Value Problem

- ▶ Consider the one-dimensional *Initial Value Problem* takes the form:
(Note the notation change from g_t to $g(t)$).

$$\partial_t g(t) = f(g(t)), \quad t > 0$$

- ▶ subject to the initial condition:

$$g(0) = \psi$$

- ▶ NOTE: We can reduce higher order equations to a system of first order equations

Analytical Solution

- ▶ Discretise time dimension with uniform time step k :

$$\{t_0, t_1, t_2, t_3, \dots, t_n, \dots\}, \quad t_0 = 0, \quad t_n = nk$$

- ▶ Discretise $g(t)$: (using the notation $G^n = g(t_n)$)

$$\{G^0, G^1, \dots, G^N, \dots\} = \{g(t_0), g(t_1), \dots, g(t_N), \dots\}$$

Basic Approaches

- ▶ Initial condition: $G_0 = \psi$. Many different algorithms for iterating forward in time:

- ▶ **Forward Euler Method:** uses forward difference leading to an *explicit scheme*:

$$\frac{G^{n+1} - G^n}{k} = f(G^n) \quad \Rightarrow \quad G^{n+1} = G^n + kf(G^n), \quad n = 1, 2, \dots$$

- ▶ **Backward Euler Method:** uses backward difference leading to an *implicit scheme*:

$$\frac{G^{n+1} - G^n}{k} = f(G^{n+1}) \quad \Rightarrow \quad G^{n+1} = G^n + kf(G^{n+1}), \quad n = 1, 2, \dots$$

- ▶ **Trapezoidal Method:** Averages the two Euler methods:

$$\frac{G^{n+1} - G^n}{k} = \frac{f(G^n) + f(G^{n+1})}{2} \quad \Rightarrow \quad G^{n+1} = G^n + k\frac{1}{2} \left(f(G^n) + f(G^{n+1}) \right), \quad n = 1, 2, \dots$$

- ▶ Implicit methods may have better stability properties.

Some Other Approaches

- ▶ **Runge-Kutta Methods:** propagate a solution over a time step by combining the information from several Euler-style steps. (Slower but typically more robust)
- ▶ **Linear Multistep Methods:** Euler and Runge-Kutta methods are *one-step* methods in the sense that G^{n+1} only depends upon G^n . Linear multistep methods allow G^{n+1} to depend a longer history of past steps.
- ▶ **Predictor-Corrector Methods:** Extrapolate the solution one period ahead using the current shape of the function and then corrects the extrapolation using derivative information at the new point.

Example 4: Initial Value PDE

- ▶ Consider diffusion equation:

$$\partial_t g(x, t) = \partial_{xx} g(x, t) + f(x), \quad x \in (x_L, x_H), \quad t > 0$$

- ▶ Subject to the initial condition:

$$g(x, 0) = \eta(x)$$

- ▶ Subject to the Dirichlet boundary conditions:

$$g(x_L, t) = g_L(t)$$

$$g(x_H, t) = g_H(t)$$

- ▶ With the forcing term:

$$f(x) = e^x$$

Discretise state space

- ▶ Discretise space dimension $[x_L, x_H]$ into J equally spaced grid points with width $h = \frac{x_J - x_1}{J-1}$:

$$\{x_1, x_2, \dots, x_j, \dots, x_{J-1}, x_J\}, \quad x_1 = x_L, \quad x_J = x_H$$

- ▶ Discretise the time dimension with uniform time step k :

$$\{t_0, t_1, t_2, t_3, \dots, t_n, \dots, t_N\}, \quad t_0 = 0, \quad t_n = nk$$

- ▶ Discretise $g(x, t)$: Let $G_j^n := g(x_j, t_n)$

Forward Euler (Explicit Method)

- Discretise system as:

$$\begin{aligned}G_1^{n+1} &= g_L(t_{n+1}), \quad j = 1 \\ \frac{G_j^{n+1} - G_j^n}{k} &= \frac{1}{h^2}(G_{j-1}^n - 2G_j^n + G_{j+1}^n) + f(x_j), \quad 2 \leq j \leq J-1 \\ G_J^{n+1} &= g_H(t_{n+1}), \quad j = J\end{aligned}$$

- For $2 \leq j \leq J-1$, this gives the iteration in the time dimension:

$$G_j^{n+1} = G_j^n + \frac{k}{h^2}(G_{j-1}^n - 2G_j^n + G_{j+1}^n + h^2 f(x_j))$$

Backward Euler (Implicit Method)

- ▶ Discretise system as:

$$\begin{aligned}G_1^{n+1} &= g_L(t_{n+1}), \quad j = 1 \\ \frac{G_j^{n+1} - G_j^n}{k} &= \frac{1}{h^2}(G_{j-1}^{n+1} - 2G_j^{n+1} + G_{j+1}^{n+1}) + f(x_j), \quad 2 \leq j \leq J-1 \\ G_J^{n+1} &= g_H(t_{n+1}), \quad j = J\end{aligned}$$

- ▶ For $2 \leq j \leq J-1$, this gives the iteration in the time dimension:

$$G_j^{n+1} = G_i^n + \frac{k}{h^2}(G_{j-1}^{n+1} - 2G_j^{n+1} + G_{j+1}^{n+1} + h^2 f(x))$$

Crank-Nicolson (“Trapezoidal” Method)

- Approximate the system using (this method is also implicit):

$$\begin{aligned}G_1^{n+1} &= g_L(t_{n+1}), \quad j = 1 \\ \frac{G_j^{n+1} - G_j^n}{k} &= \frac{1}{2}(D^2 G_j^n + D^2 G_j^{n+1}) + f(x_j), \quad 2 \leq j \leq J-1 \\ &= \frac{1}{2h^2}(G_{j-1}^n - 2G_j^n + G_{j+1}^n + G_{j-1}^{n+1} - 2G_j^{n+1} + G_{j+1}^{n+1}) + f(x_j) \\ G_J^{n+1} &= g_H(t_{n+1}), \quad j = J\end{aligned}$$

- For $2 \leq j \leq J-1$, this can be rewritten as:

$$G_j^{n+1} = G_j^n + \frac{k}{2h^2}(G_{j-1}^n - 2G_j^n + G_{j+1}^n + G_{j-1}^{n+1} - 2G_j^{n+1} + G_{j+1}^{n+1} + h^2 f(x_j))$$

- or the following (where $r = k/(2h^2)$):

$$-rG_{j-1}^{n+1} + (1 + 2r)G_j^{n+1} - rG_{j+1}^{n+1} = rG_{j-1}^n + (1 - 2r)G_j^n + rG_{j+1}^n + rh^2 f(x_j)$$

Finite Difference Approximation (Version 1)

- At each step n , we can stack the equations in matrix form as before:

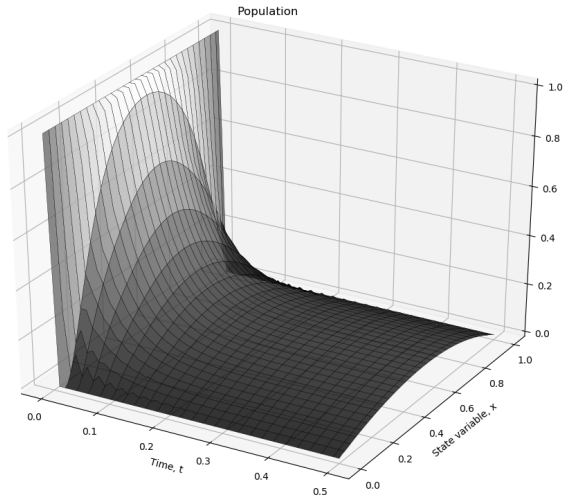
$$\underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & \dots & 0 & 0 \\ -r & (1+2r) & -r & 0 & 0 & \dots & 0 & 0 \\ 0 & -r & (1+2r) & -r & 0 & \dots & 0 & 0 \\ & & & \ddots & \ddots & \ddots & & \\ 0 & 0 & \dots & 0 & 0 & -r & (1+2r) & -r \\ 0 & 0 & \dots & 0 & 0 & 0 & 0 & 1 \end{bmatrix}}_{:=A} \underbrace{\begin{bmatrix} G_1^{n+1} \\ G_2^{n+1} \\ G_3^{n+1} \\ \vdots \\ G_{J-1}^{n+1} \\ G_J^{n+1} \end{bmatrix}}_{:=G^{n+1}} \\
 = \underbrace{\begin{bmatrix} g_L^{n+1} \\ rG_1^n + (1-2r)G_2^n + rG_3^n + rh^2f(x_2) \\ rG_2^n + (1-2r)G_3^n + rG_4^n + rh^2f(x_3) \\ \vdots \\ rG_{J-2}^n + (1-2r)G_{J-1}^n + rG_J^n + rh^2f(x_{J-1}) \\ u_H^{n+1} \end{bmatrix}}_{:=F(G^n)}$$

Algorithm

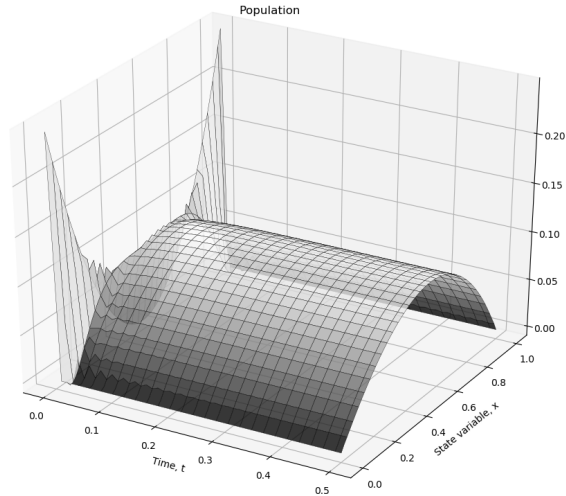
- ▶ For $n = 0$, start with the initial density $G^0 = (G_1^0, \dots, G_J^0)$
- ▶ For each $n > 0$, solve:

$$G^{n+1} = A^{-1}F(G^n)$$

Sample Code ($g(x, 0) = \eta(x) = 1$)



Sample Code ($g(x, 0) = \eta(x) = (x - 0.5)^2$)



Q: What are some potential problems with this technique?

Q: Could we use this technique for a differential equation with 100 dimensions?

Q/A

Q: What are some potential problems with this technique?

- ▶ **A.** We are approximating derivatives with slopes between grid points.
- ▶ If the problem has high curvature, then we need a very fine grid, which is computationally hard
- ▶ Alternative is to project V onto basis functions rather than grid points.

Q: Could we use this technique for a differential equation with 100 dimensions?

- ▶ **A.** No chance!
- ▶ If we have N grid points in very dimension, then total matrix dimension is N^{100} .
- ▶ Physics literature has turned to deep learning to solve high dimensional problems.
- ▶ The idea is to represent V by a neural network and then train parameters so neural network solves the differential equation.

Table of Contents

An Test Model of Population Evolution

Finite Difference

Example 1: Steady State KFE With Exponential Entry Rate

Example 2: Neumann Boundary Conditions

Example 4: Initial Value Problem

Artificial Neural Networks

Introduction to Neural Nets

Solving Differential Equations With Neural Networks

Numerical Packages

Numerical Examples

Optional Appendix: Spectral Method

General Principles

Example 1: Linear ODE

Example 4: (Revisited)

Table of Contents

An Test Model of Population Evolution

Finite Difference

Example 1: Steady State KFE With Exponential Entry Rate

Example 2: Neumann Boundary Conditions

Example 4: Initial Value Problem

Artificial Neural Networks

Introduction to Neural Nets

Solving Differential Equations With Neural Networks

Numerical Packages

Numerical Examples

Optional Appendix: Spectral Method

General Principles

Example 1: Linear ODE

Example 4: (Revisited)

Multi-Layer Perceptrons (MLPs)

- ▶ An MLP attempts to approximate some function $\mathbf{y} = f^*(\mathbf{x})$, where $\mathbf{x}$ and $\mathbf{y}$ are vectors of dimension p and q
- ▶ Approximation uses a “network” structure:
 - ▶ Approximates f^* by a mapping $y = f(\mathbf{x}, \theta)$, with parameters θ
 - ▶ The mapping, f , is typically represented by composing many different functions in a chain (i.e. a *network structure*)
- ▶ Other names: *feedforward* or *deep feedforward neural networks*
- ▶ Ultimately, we will “learn” the parameters, θ , that give “best” approximation

General MLP Structure

- ▶ In general, an MLP neural network approximation takes the form:

$$\mathbf{h}^{(1)} = \phi^{(1)}(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) \quad \dots \text{Hidden layer 1}$$

$$\mathbf{h}^{(2)} = \phi^{(2)}(W^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}) \quad \dots \text{Hidden layer 2}$$

$$\vdots$$

$$\mathbf{h}^{(H)} = \phi^{(H)}(W^{(H)}\mathbf{h}^{(H-1)} + \mathbf{b}^{(H)}) \quad \dots \text{Hidden layer H}$$

$$\mathbf{o} = W^{(H+1)}\mathbf{h}^{(H)} + \mathbf{b}^{(H+1)} \quad \dots \text{Output layer}$$

$$\mathbf{y} = \sigma(\mathbf{o}) \quad \dots \text{Output}$$

- ▶ Terminology:

- ▶ H : is the number of *hidden layers*
- ▶ Length of vector $\mathbf{h}^{(i)}$: number of *neurons* in hidden layer i
- ▶ $\phi^{(i)}$: is the *activation function* for hidden layer i
- ▶ σ : is the *activation function* for the output layer
- ▶ $\theta = (W^1, \dots, W^{(H+1)}, b^{(1)}, \dots, b^{(H+1)})$ are the *parameters*

Ex 1: No Hidden Layers, Linear Network

- ▶ Suppose that $H = 0$ (no hidden layers); $\mathbf{x} \in \mathbb{R}^4$; $\mathbf{y} \in \mathbb{R}^3$
- ▶ In this case, the neural network is just a linear combination:
 - ▶ In matrix notation: $\mathbf{o} = W\mathbf{x} + \mathbf{b}$ and $\mathbf{y} = \mathbf{o}$

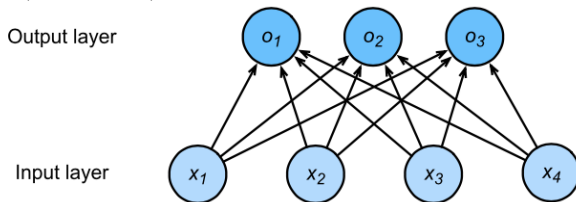
- ▶ Without matrix notation:

$$o_1 = x_1W_{11} + x_2W_{12} + x_3W_{13} + x_4W_{14} + b_1, \quad y_1 = o_1$$

$$o_2 = x_1W_{21} + x_2W_{22} + x_3W_{23} + x_4W_{24} + b_2, \quad y_2 = o_2$$

$$o_3 = x_1W_{31} + x_2W_{32} + x_3W_{33} + x_4W_{34} + b_3, \quad y_3 = o_3$$

- ▶ Graphical depiction (often used):



- ▶ This is called a *dense* neural network because it is “fully connected”

Ex 2: One Hidden Layer; Linear Network

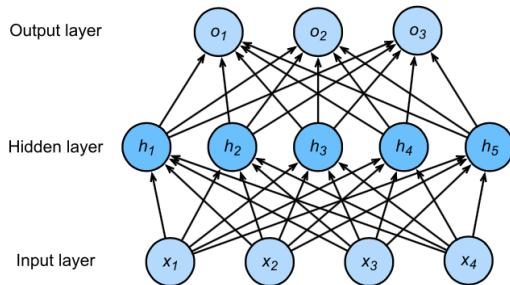
- Suppose $H = 1$ (one hidden layer with 5 neurons); $\mathbf{x} \in \mathbb{R}^4$; $\mathbf{y} \in \mathbb{R}^3$
- Neural network (in matrix notation):

$$\mathbf{h} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, \quad W^{(1)} \text{ is } 5 \times 4, \quad \mathbf{b}^{(1)} \text{ is } 5 \times 1$$

$$\mathbf{o} = W^{(2)}\mathbf{h} + \mathbf{b}^{(2)}, \quad W^{(2)} \text{ is } 3 \times 5, \quad \mathbf{b}^{(2)} \text{ is } 3 \times 1$$

$$\mathbf{y} = \mathbf{o}$$

- Neural network (graphically):



Did the hidden layer add anything?

Adding Linear Layers

- ▶ Introducing additional linear layers doesn't add anything
- ▶ Why? Because the composition of linear layers is still linear
- ▶ Instead, we need to add non-linear activation functions to generate more complicated approximations

Ex 3: One Hidden Layer; Non-Linear Network

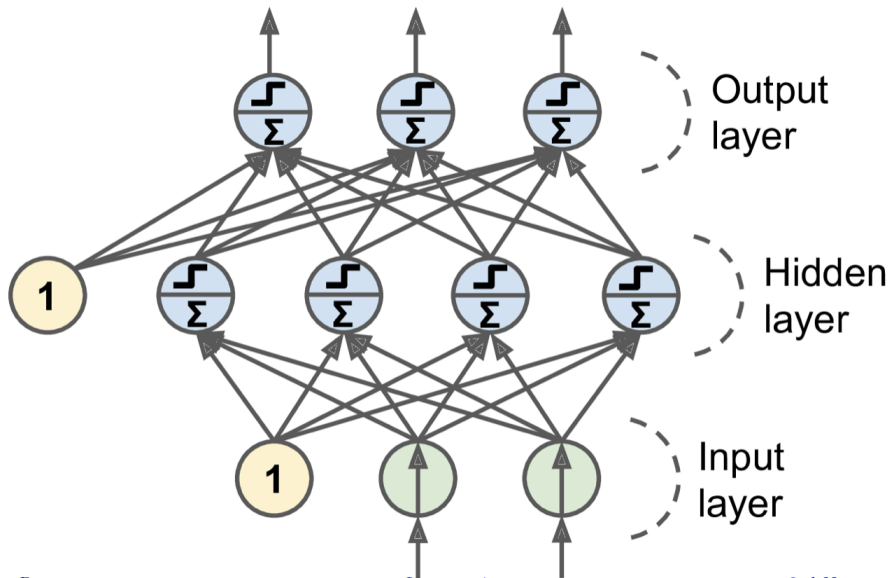
- ▶ Suppose $H = 1$ (one hidden layer with 4 neurons); $\mathbf{x} \in \mathbb{R}^2$; $\mathbf{y} \in \mathbb{R}^3$
- ▶ Neural network (in matrix notation):

$$\begin{aligned}\mathbf{h} &= \phi(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), & W^{(1)} \text{ is } 4 \times 2, \quad \mathbf{b}^{(1)} \text{ is } 4 \times 1 \\ \mathbf{o} &= W^{(2)}\mathbf{h} + \mathbf{b}^{(2)}, & W^{(2)} \text{ is } 3 \times 4, \quad \mathbf{b}^{(2)} \text{ is } 3 \times 1 \\ \mathbf{y} &= \mathbf{o}\end{aligned}$$

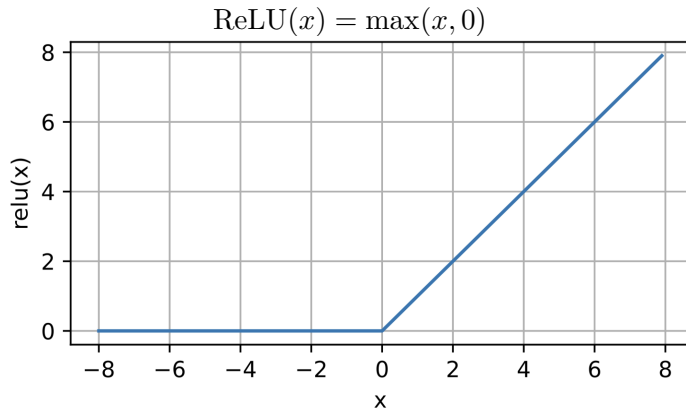
where the non-linear *activation function* ϕ is applied to each element of the vector $W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$

- ▶ Common ϕ functions:
 - ▶ Rectified linear unit (ReLU) function
 - ▶ Sigmoid function
 - ▶ Tanh function

Aside: Geron (2019)'s Graphical Depiction

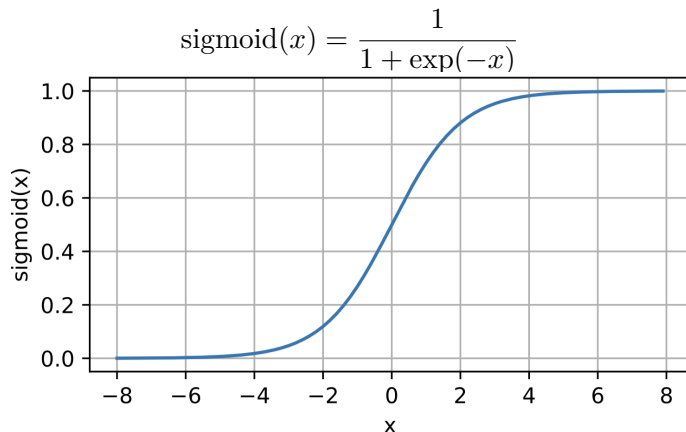


Activation Function: ReLU



- ▶ The Rectified Linear Unit (ReLU) function retains only positive elements
- ▶ Informally, the ReLU function retains only positive elements

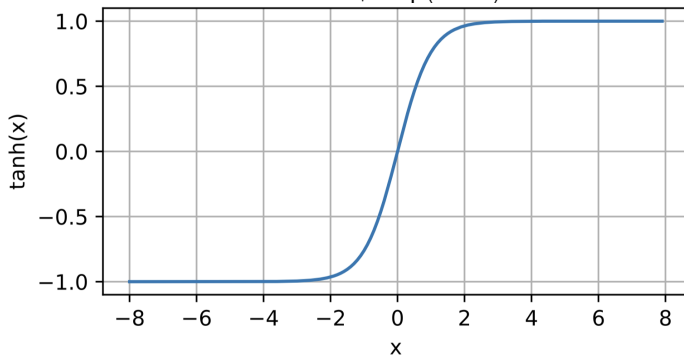
Activation Function: Sigmoid Function



- ▶ Transforms (“squashes”) the values into the interval (0, 1)
- ▶ Helpful if the function f is smooth

Activation Function: Tanh Function

$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}$$



- ▶ The tanh (Hyperbolic Tangent) also squashes its inputs into $[-1, 1]$
- ▶ The tanh function exhibits point symmetry about the origin

Can a Deep Neural Network approximate a wide range of functions?

Universal Approximation Theorem

Theorem (Universal Approximation Theorem (UAT))

An MLP with a linear output layer and at least one hidden layer with any “squashing” activation function (e.g. logistic sigmoid activation function) can approximate any Borel measurable function from one finite-dimensional space to another with any desired non-zero amount of error, provided that the network is given enough hidden units.

Proof.

See Goodfellow (2016). $\square$

- ▶ i.e. A sufficiently large MLP can represent “any” function,
- ▶ However, some notes of caution:
 - (i) Theoretical conditions on the UAT don’t always hold,
 - (ii) We are not guaranteed that we will be able to train our MLP to find the true function

Table of Contents

An Test Model of Population Evolution

Finite Difference

Example 1: Steady State KFE With Exponential Entry Rate

Example 2: Neumann Boundary Conditions

Example 4: Initial Value Problem

Artificial Neural Networks

Introduction to Neural Nets

Solving Differential Equations With Neural Networks

Numerical Packages

Numerical Examples

Optional Appendix: Spectral Method

General Principles

Example 1: Linear ODE

Example 4: (Revisited)

Approaches

- ▶ Train neural network to minimize error in differential equation: (This lecture).
 - ▶ Raissi-Perdikaris-Karniadakis (2017). *“Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations.”* (Physics Informed Neural Networks (PINNs))
 - ▶ Sirignano-Spiliopoulos (2018). *“Deep Galerkin Method (DGM): A deep learning algorithm for solving partial differential equations”*.
 - ▶ Gu-Lauriere-Merkel-Payne (2023). *“Deep Learning Solutions to Master Equations for Continuous Time Heterogeneous Agent Macroeconomic Models”*
- ▶ Simulate the economy and learn policies that minimize errors along average path:
 - ▶ Han, Jentzen, and Weinan (2018). *“Solving high-dimensional partial differential equations using deep learning”*
 - ▶ Huang (2022)). *“A probabilistic solution to high-dimensional continuous-time macro-finance models.”*

A General (Quasilinear Parabolic) PDE System

- Consider the (time independent quasilinear parabolic) PDE system:

$$\begin{aligned}\mathcal{L}V(x) &= 0, & x \in \mathcal{X} \\ V(x) &= b(x), & x \in \partial\mathcal{X}\end{aligned}$$

where

- $\mathcal{L}$ is a differential operator. E.g.

$$\mathcal{L}V(x) = \partial_{xx}V(x) + \partial_xV(x)$$

- $\partial\mathcal{X}$ is the boundary of the domain $\mathcal{X}$
- $b(x)$ is the boundary condition

Solution Approach: Idea

- ▶ Approximates the function $V(x)$ with a deep neural network $\hat{V}(x; \theta)$, where $\theta \in \mathbb{R}^K$ are the neural network's parameters.
- ▶ Goal: choose parameters, θ , to minimise the loss function:

$$\mathcal{E}(\hat{V}) = \left\| \mathcal{L}\hat{V}(x; \theta) \right\|_{\mathcal{X}, \nu_1}^2 + \left\| \hat{V}(x; \theta) - b(x) \right\|_{\mathcal{X}, \nu_2}^2$$

- ▶ The norm is given by $\|f(x)\|_{\mathcal{X}, \nu}^2 := \int_{\mathcal{X}} |f(x)|^2 \nu(x) dx$ where $\nu(x)$ is a positive probability density on $x \in \mathcal{X}$.

In practice, we need to approximate the norms by sampling from the state space.

Algorithm

Starting with an initial guess for the parameters θ^0 . At iteration n with guess θ^n :

1. Sample $S^n := \{S^{ni}, S^{nb}\}$ from the state space, where:

- ▶ $S^{ni} = \{x_m\}_{m \leq M^e}$ are sample points on **interior** $x_m \in \mathcal{X}$,
- ▶ $S^{nb} = \{x_m\}_{m \leq M^b}$ are sample points on **boundary** $x_m \in \partial\mathcal{X}$.

2. Calculate the **weighted average error** on the sample:

$$\mathcal{E}(\theta^n, S^n) = \kappa^i \mathcal{E}^i(\theta^n, S^{ni}) + \kappa^b \mathcal{E}^b(\theta^n, S^{nb}) + \kappa^f \mathcal{E}^f(\theta^n, S^{ni}), \quad \text{where}$$

- ▶ $\mathcal{E}^i(\theta^n, S^{ni}) := \frac{1}{M^e} \sum_{m \leq M^e} |\mathcal{L}\hat{V}(x_m)|^2$ is **error in differential equation** on sample S^{ni}
- ▶ $\mathcal{E}^b(\theta^n, S^{nb})$ is **error on boundary**
- ▶ $\mathcal{E}^f(\theta^n, S^{ni})$ is **penalty for “wrong” shape** (e.g. penalty for non-concavity of V)
- ▶ Derivatives are evaluated using **automatic differentiation**.

3. **Update the NN parameters** using a variation on “stochastic” gradient descent:

$$\theta^{n+1} = \theta^n - \alpha_n D_\theta \mathcal{E}(\theta^n, S^n)$$

Great in Principle But Implementation is Tricky

- ▶ Algorithm has some large advantages for continuous time:
 1. Can deal with high dimensional differential equations,
 2. Calculates derivatives using automatic differentiation (rather than finite difference),
 3. Can sample on random points rather instead of grid.
- ▶ However, the algorithm is tricky to implement (despite being easy describe)!
- ▶ We need to focus on implementation details (particularly for high dimensions).

Neural Network Q & A

- ▶ *Q. Why do we draw new samples each epoch?*
 - ▶ Avoid **overfitting** on random samples.
- ▶ *Q. Could we use an alternative parametric approx. (e.g. Chebyshev polynomials)?*
 - ▶ In principle, yes.
 - ▶ However, **automatic derivatives** can easily be calculated for neural networks.
 - ▶ and effective **non-linear optimizers** have been developed for neural nets.
- ▶ *Q. Does the algorithm solve for global minimum?*
 - ▶ Loss calculated by **randomly collected points** so we move around the parameter space.
 - ▶ Helps to escape from local minimum trap by SGD (but does not seem to prevent it).

Neural Network Q & A

- ▶ *Q. Why do we need shape constraints?*
 - ▶ Neural network can find “bad” approximate solutions,
 - ▶ Especially likely to find solutions with zero derivative when limited curvature in the problem or dimension is very high,
 - ▶ Forcing a shape constraint can prevent this.
- ▶ *Q. What if there is a time dimension and an initial condition?*
 - ▶ We need to add an additional loss for the initial condition.
- ▶ *Q. Why are neural networks better for high dimensional PDEs?*
 - ▶ Because we can train them on incrementally of arbitrarily sized samples.
 - ▶ That is, we never need to write down a grid.

Algorithm Typical “Hyper-Parameter” Choices

- ▶ Number of layers (typically 1 – 5)
- ▶ Number of neurons per layer (typically 10 – 100)
- ▶ Number of neurons in the output layer (1 per classification class)
- ▶ Hidden activation function (I often use the Sigmoid or Tanh)
- ▶ Output activation (none; log-odds; soft-max)

Table of Contents

An Test Model of Population Evolution

Finite Difference

Example 1: Steady State KFE With Exponential Entry Rate

Example 2: Neumann Boundary Conditions

Example 4: Initial Value Problem

Artificial Neural Networks

Introduction to Neural Nets

Solving Differential Equations With Neural Networks

Numerical Packages

Numerical Examples

Optional Appendix: Spectral Method

General Principles

Example 1: Linear ODE

Example 4: (Revisited)

Practical Requirements

- ▶ There are technical challenges to working with neural nets
(which initially stalled the development)
- ▶ Challenge 1: Need to find θ that minimises the loss function
 - ▶ Requires an efficient non-linear optimization package
 - ▶ Typically uses stochastic gradient descent
- ▶ Challenge 2: Need to differentiate the neural network
 - ▶ Requires an “automatic differentiation” package that can calculate derivatives efficiently
 - ▶ Advances in automatic differentiation have made deep learning feasible
(just as they have revolutionised many other areas)

Some Packages

- ▶ PyTorch* (what I use)
- ▶ TensorFlow
- ▶ JAX (what is popular at the moment)
- ▶ Scikit learn
- ▶ Mxnet
- ▶ Caffe
- ▶ CNTK
- ▶ Flux

References on PyTorch

- ▶ PyTorch official tutorials (<https://pytorch.org/tutorials/>) is the best material to start with. The 60-minute blitz provides an overview into constructing deep neural networks with PyTorch.
- ▶ PyTorch Docs (<https://pytorch.org/docs/stable/index.html>) serves as the first-hand material for understanding the API, including Torch.nn (<https://pytorch.org/docs/stable/nn.htmz>), Torch.optim (<https://pytorch.org/docs/stable/optim.html>)
- ▶ PyTorch Community (<https://discuss.pytorch.org/>) is the “Slack overflow” or “MathWorks” of PyTorch.
- ▶ Justin Johnson has a very simple and clean note for PyTorch, which can help you get familiar with all the essentials. (<https://github.com/jcjohnson/pytorch-examples>.)

References on TensorFlow (version 2)

- ▶ TensorFlow website: <https://www.tensorflow.org>
- ▶ TensorFlow has tutorials (<https://www.tensorflow.org/tutorials/>) and a guide (<https://www.tensorflow.org/guide>) on their website. The Keras guide is a good place to start learning the basic Tensorflow syntax.
- ▶ Geron (2019): Has been updated for Keras and TensorFlow 2
- ▶ Chollet (2018): This book introduces the basics of deep learning with Keras (but still uses TensorFlow version 1).

Table of Contents

An Test Model of Population Evolution

Finite Difference

Example 1: Steady State KFE With Exponential Entry Rate

Example 2: Neumann Boundary Conditions

Example 4: Initial Value Problem

Artificial Neural Networks

Introduction to Neural Nets

Solving Differential Equations With Neural Networks

Numerical Packages

Numerical Examples

Optional Appendix: Spectral Method

General Principles

Example 1: Linear ODE

Example 4: (Revisited)

Example 1: Ordinary Differential Equation

- ▶ Consider the differential equation:

$$\begin{aligned}y'(x) &= -2xy, & x \in \mathbb{R} \\ y(0) &= 1\end{aligned}$$

- ▶ Sample code:
 - ▶ Tensorflow: `ode_solver_ex01.py`
 - ▶ PyTorch: `ode_solver_ex01_pytorch.py`

Example 2: Ordinary Differential Equation

- ▶ Consider the differential equation (steady state diffusion / 1D Poisson)

$$0 = \partial_{xx}g(x) + f(x), \quad x \in (x_L, x_H)$$

- ▶ Subject to the Dirichlet boundary conditions:

$$g(x_L) = g_L$$

$$g(x_H) = g_H$$

- ▶ And the forcing term:

$$f(x) = e^x$$

- ▶ Sample code:

- ▶ Tensorflow: `ode_solver_ex02_loop.py` (manual loop)
- ▶ Tensorflow: `ode_solver_ex02_fit.py` (using Tensorflow fit)
- ▶ PyTorch: `ode_solver_ex02_pytorch.py`

References I



Candler, G. V. (2001).

Finite-Difference Methods for Continuous-Time Dynamic Programming.



Géron, A. (2019).

Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems.

O'Reilly Media.



Goodfellow, I., Bengio, Y., and Courville, A. (2016).

Deep learning.

MIT press.



Holmes, M. H. (2007).

Introduction to Numerical Methods in Differential Equations, volume 44.

Springer.



Iserles, A. (2008).

A First Course in the Numerical Analysis of Differential Equations.

Cambridge University Press.



LeVeque, R. J. (2007).

Finite Difference Methods for Ordinary and Partial Differential Equations.

References II



Pichler, L., Masud, A., and Bergman, L. (2013).

Numerical Solution of the Fokker–Planck Equation by Finite Difference and Finite Element Methods—A Comparative Study.



Pozrikidis, C. (2005).

Finite and Spectral Element Methods using MATLAB.

Chapman and Hall.



Press, W. H., Flannery, B. P., Teukolsky, S. A., and Wetterling, W. (1988).

Numerical recipes in C.

Press Syndicate of the University of Cambridge.



Trefethen, L. N. (1996).

Finite Difference and Spectral Methods for Ordinary and Partial.



Trefethen, L. N. (2001).

Spectral Methods in MATLAB.



Zhang, A., Lipton, Z. C., Li, M., and Smola, A. J. (2021).

Dive into deep learning.

arXiv preprint arXiv:2106.11342.

Table of Contents

An Test Model of Population Evolution

Finite Difference

Example 1: Steady State KFE With Exponential Entry Rate

Example 2: Neumann Boundary Conditions

Example 4: Initial Value Problem

Artificial Neural Networks

Introduction to Neural Nets

Solving Differential Equations With Neural Networks

Numerical Packages

Numerical Examples

Optional Appendix: Spectral Method

General Principles

Example 1: Linear ODE

Example 4: (Revisited)

Table of Contents

An Test Model of Population Evolution

Finite Difference

Example 1: Steady State KFE With Exponential Entry Rate

Example 2: Neumann Boundary Conditions

Example 4: Initial Value Problem

Artificial Neural Networks

Introduction to Neural Nets

Solving Differential Equations With Neural Networks

Numerical Packages

Numerical Examples

Optional Appendix: Spectral Method

General Principles

Example 1: Linear ODE

Example 4: (Revisited)

Collocation Techniques

- ▶ Collocation techniques approximate a function $g(x)$ by a linear combination of a finite set of orthogonal basis functions, $\{\varphi_j(x)\}_{1 \leq j \leq N}$:

$$g(x) = \sum_{j=1}^N c_j \varphi_j(x), \quad \text{where } \{c_j\}_{j \leq N} \text{ are coefficients for functions } \{\varphi_j(x)\}_{j \leq N}$$

- ▶ **Galerkin approach:** Chose coefficients to minimize an appropriate norm of the residual function from the differential equation (e.g. $\|\partial_{xx}g(x) - f(x)\|$).
- ▶ **Spectral collocation:** Chose coefficients s.t. the differential equation holds at selected grid points and boundary conditions (e.g. $g''(x_i) = f(x_i)$ on grid $\partial_{xx}\{x_i\}_{i \leq I}$).
- ▶ This section outlines a simple version of the spectral collocation approach using Chebyshev polynomials.
- ▶ Next section uses Neural Networks.

Discretisation of State Space

- ▶ Consider a function $g(x)$ on $-1 \leq x \leq 1$.
- ▶ Discretise state space: choose $N + 1$ grid points: (The particular points chosen below are called the **Chebyshev points**. See Trefethen (2001), chapter 5 for a discussion of why these points are optimal.)

$$\{x_0, x_1, \dots, x_N\}, \quad x = \cos(j\pi/N), \quad j = 0, 1, \dots, N.$$

- ▶ Denote the approximation of g on the grid points by:

$$G := \{G_0, G_1, \dots, G_N\} := \{g(x_0), g(x_1), \dots, g(x_N)\}$$

- ▶ We want to approximate the derivatives ($\partial_x g$, $\partial_{xx} g$, ect.) on the grid points

Approximating Derivatives

- ▶ Denote the approximation of g on $j \in \{0, 1, \dots, N\}$ by $G := \{G_0, G_1, \dots, G_N\}$.
- ▶ Approximating $\partial_x g$ on $j \in \{0, 1, \dots, N\}$ involves two steps:
 1. Find the unique polynomial, p , of degree $\leq N$ that satisfies $p(x_j) = G_j$, $0 \leq j \leq N$. (i.e. the unique polynomial that fits each grid point).
 2. For each $j \in \{0, 1, \dots, N\}$, set $w_j = p'(x_j)$. This is a linear operation and so generates a differentiation matrix, D_N satisfying:

$$\hat{\partial}_x \underline{g} = \underline{w} = D_N \underline{G}, \quad \hat{\partial}_x \underline{g} = \begin{bmatrix} \partial_x g(x)|_{x=x_1} \\ \vdots \\ \partial_x g(x)|_{x=x_N} \end{bmatrix}, \quad \underline{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_N \end{bmatrix}, \quad \underline{G} = \begin{bmatrix} G_1 \\ \vdots \\ G_N \end{bmatrix}$$

- ▶ The approximation for $\partial_{xx} g$ on $j \in \{0, 1, \dots, N\}$ is then given by:

$$\hat{\partial}_{xx} \underline{g} = D_N^2 \underline{G}$$

Differentiation Matrix: Example $N = 1$

- Consider the case with 2 grid points. In this case, the interpolation points are:

$$x_0 = \cos(0) = 1, \quad x_1 = \cos(\pi/1) = -1$$

- Let $G = \{G_0, G_1\}$ denote the approximation of $g(x)$ on $\{x_0, x_1\} = \{1, -1\}$.
- Step 1: The interpolating polynomial (satisfying $p(x_0) = G_0$ and $p(x_1) = G_1$) is:

$$p(x) = \frac{1}{2}(1+x)G_0 + \frac{1}{2}(1-x)G_1$$

- Step 2: Taking the derivative gives:

$$p'(x) = \frac{1}{2}G_0 - \frac{1}{2}G_1$$

Differentiation Matrix: Example $N = 1$

- So, the matrix representation for the approximation to $\partial_x g$ is:

$$\hat{\partial}_x g = \begin{bmatrix} \hat{\partial}_x g(x_0) \\ \hat{\partial}_x g(x_1) \end{bmatrix} = \begin{bmatrix} \frac{1}{2}G_0 - \frac{1}{2}G_1 \\ \frac{1}{2}G_0 - \frac{1}{2}G_1 \end{bmatrix} = D_1 \underline{G},$$

where

$$D_1 = \frac{1}{2} \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}, \quad \underline{G} = \begin{bmatrix} G_0 \\ G_1 \end{bmatrix}$$

Differentiation Matrix: Example $N = 2$

- Consider the case with 3 grid points ($N = 2$).

- In this case, the interpolation points:

$$x_0 = 1, \quad x_1 = 0, \quad x_2 = -1$$

- Let $G = \{G_0, G_1, G_2\}$ denote the approximation of $g(x)$ on $\{x_0, x_1, x_2\} = \{1, 0, -1\}$.

- Step 1: The interpolating polynomial (satisfying $p(x_0) = G_0$, $p(x_1) = G_1$, and $p(x_2) = G_2$) is:

$$p(x) = \frac{1}{2}x(1+x)G_0 + (1+x)(1-x)G_1 + \frac{1}{2}x(x-1)G_2$$

- Step 2: Taking the derivative gives:

$$p'(x) = \left(x + \frac{1}{2}\right) G_0 - 2xG_1 + \left(x - \frac{1}{2}\right) G_2$$

Differentiation Matrix: Example $N = 2$

► And so the derivative matrix is:

$$\hat{\partial}_x g = \begin{bmatrix} \hat{\partial}_x g(x_0) \\ \hat{\partial}_x g(x_1) \\ \hat{\partial}_x g(x_2) \end{bmatrix} = \begin{bmatrix} \frac{3}{2}G_0 - 2G_1 + \frac{1}{2}G_2 \\ \frac{1}{2}G_0 - \frac{1}{2}G_2 \\ -\frac{1}{2}G_0 + 2G_1 - \frac{3}{2}G_2 \end{bmatrix} = D_2 \underline{G}$$

where

$$D_2 = \frac{1}{2} \begin{bmatrix} 3 & -4 & 1 \\ 1 & 0 & -1 \\ -1 & 4 & -3 \end{bmatrix}, \quad \underline{G} = \begin{bmatrix} G_0 \\ G_1 \\ G_2 \end{bmatrix}$$

Differentiation Matrix: General

- **Theorem:** Let $\{x_0, x_1, \dots, x_N\}$ denote the Chebyshev points for an $N + 1$ dimensional grid. For each $N \geq 1$, let the rows and columns of the $(N + 1) \times (N + 1)$ Chebyshev spectral differentiation matrix D_N be indexed from 0 to N . The entries of the matrix are:

$$[D_N]_{00} = \frac{2N^2 + 1}{6}, \quad [D_N]_{NN} = -\left(\frac{2N^2 + 1}{6}\right)$$

$$[D_N]_{jj} = \frac{-x_j}{2(1 - x_j^2)}, \quad j = 1, \dots, N - 1$$

$$[D_N]_{ij} = \frac{c_i}{c_j} \frac{(-1)^{i+j}}{(x_i - x_j)}, \quad i \neq j, \quad i, j = 1, \dots, N - 1$$

Differentiation Matrix: General Graphical Depiction

$D_N =$

$\frac{2N^2 + 1}{6}$	$2 \frac{(-1)^j}{1 - x_j}$	$\frac{1}{2}(-1)^N$
$-\frac{1}{2} \frac{(-1)^i}{1 - x_i}$	$\frac{(-1)^{i+j}}{x_i - x_j}$ $\frac{-x_j}{2(1 - x_j^2)}$ $\frac{(-1)^{i+j}}{x_i - x_j}$	$\frac{1}{2} \frac{(-1)^{N+i}}{1 + x_i}$
$-\frac{1}{2}(-1)^N$	$-2 \frac{(-1)^{N+j}}{1 + x_j}$	$-\frac{2N^2 + 1}{6}$

Table of Contents

An Test Model of Population Evolution

Finite Difference

Example 1: Steady State KFE With Exponential Entry Rate

Example 2: Neumann Boundary Conditions

Example 4: Initial Value Problem

Artificial Neural Networks

Introduction to Neural Nets

Solving Differential Equations With Neural Networks

Numerical Packages

Numerical Examples

Optional Appendix: Spectral Method

General Principles

Example 1: Linear ODE

Example 4: (Revisited)

Example 1 (Revisited)

- ▶ Consider the differential equation (steady state diffusion / 1D Poisson)

$$0 = f(x) + \partial_{xx}g(x), \quad x \in (-1, 1)$$

- ▶ subject to the Dirichlet boundary conditions:

$$g(x_L) = g_L = 0$$

$$g(x_H) = g_H = 0$$

- ▶ and the forcing term:

$$f(x) = e^x$$

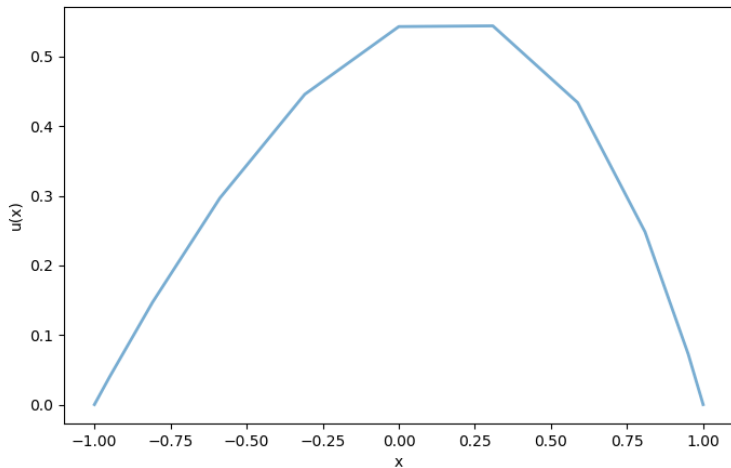
Spectral Approximation to Differential Equation

- ▶ Let D_N be the N-grid point spectral approximation to ∂_x and let $D_N^2 = D_N D_N$ be the N-grid point spectral approximation to $\partial_x x$. Then, the DE is:

$$\begin{aligned} G_1 &= 0, \quad j = 1 \\ [D_N^2]_{(j,\cdot)} \cdot G &= f(x_j), \quad j = 2, \dots, N-1 \\ G_N &= 0, \quad j = N \end{aligned}$$

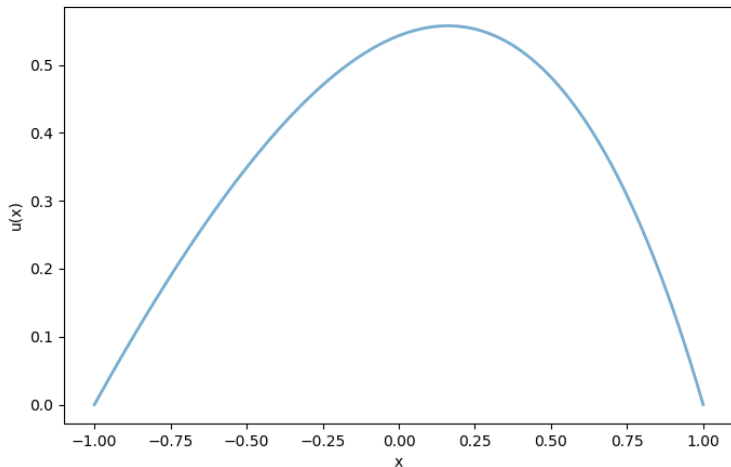
- ▶ So, for this problem, the outermost rows and columns in D_N^2 play no role and so can be excluded.

Sample Code (No Smoothing)



Julia/SM_Pois1DEqnSolver.jl

Sample Code (Polynomial Smoothing)



Julia/SM_Pois1DEqnSolver.jl

Problems on Other Domains

- ▶ What if we want to solve a problem on $[a, b]$ not on $x \in [-1, 1]$.
- ▶ **Option 1:** Find new optimal Chebyshev points:

$$x_i = a + \frac{1}{2}(b - a)(1 + \cos(\pi(1 - z_i))), \quad \text{for } i = 0, 1, \dots, N$$

where z_i are evenly spaced in the unit interval: $z_i = i/(N)$, for $i = 0, 1, \dots, N$. Then, find the matrices for those points.

- ▶ **Option 2:** Use change of variables in the problem to get domain into $[-1, 1]$

Table of Contents

An Test Model of Population Evolution

Finite Difference

Example 1: Steady State KFE With Exponential Entry Rate

Example 2: Neumann Boundary Conditions

Example 4: Initial Value Problem

Artificial Neural Networks

Introduction to Neural Nets

Solving Differential Equations With Neural Networks

Numerical Packages

Numerical Examples

Optional Appendix: Spectral Method

General Principles

Example 1: Linear ODE

Example 4: (Revisited)

Example 1 (Revisited)

- ▶ Consider the differential equation (steady state diffusion / 1D Poisson)

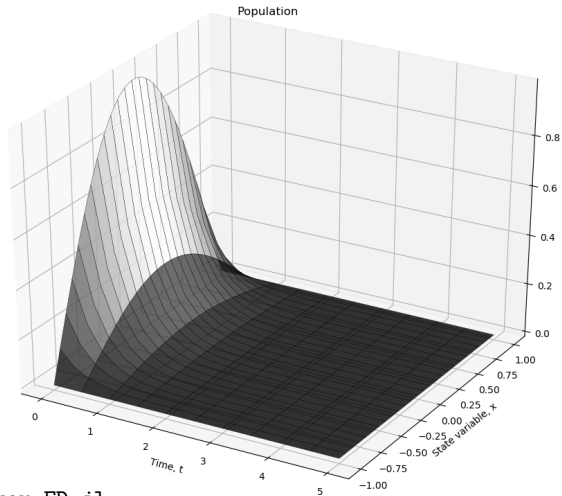
$$0 = f(x) + \partial_{xx}g(x), \quad x \in (-1, 1)$$

- ▶ subject to the Dirichlet boundary conditions:

$$g(x_L) = g_L = 0$$

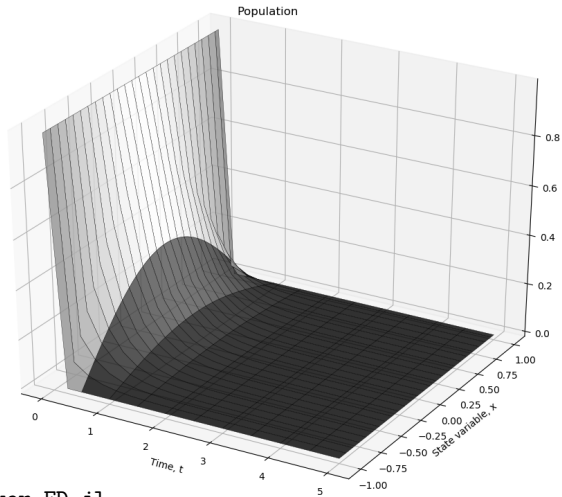
$$g(x_H) = g_H = 0$$

Sample Code ($g(x, 0) = \cos(x\pi/2)$, $f(x) = 0$)



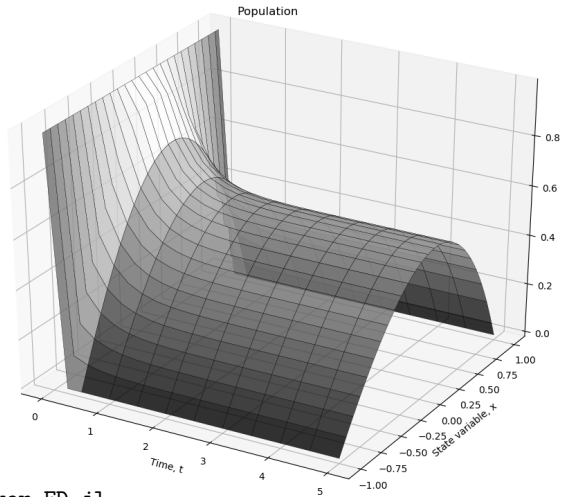
Julia/FD_Ex04_HeatEqnSolver_FD.jl

Sample Code ($g(x, 0) = 1, f(x) = 0$)



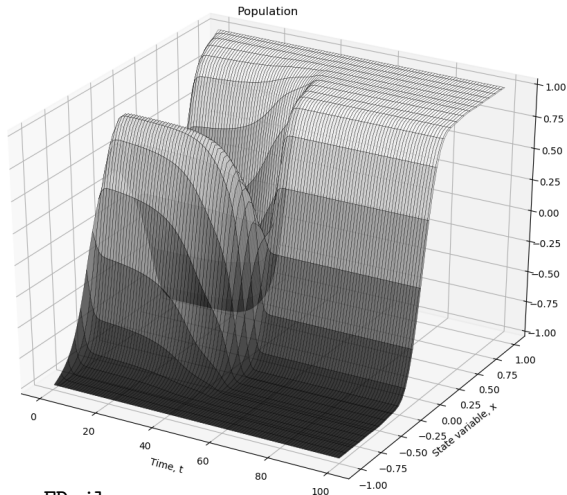
Julia/FD_Ex04_HeatEqnSolver_FD.jl

Sample Code ($g(x, 0) = 1, f(x) = 0$)



Julia/FD_Ex04_HeatEqnSolver_FD.jl

Sample Code ($g(x, 0) = .53x + .47 \sin(-1.5\pi x)$, $f(x) = x - x^3$)



Julia/FD_Ex04_HeatEqnSolver_FD.jl

Q: What are some potential problems with this technique?

Q: Could we use this technique for a differential equation with 100 dimensions?

Q: What are some potential problems with this technique?

- ▶ **A.** The method can better handle curvature but still needs a grid.
- ▶ We need to move towards random sampling (i.e. the Galerkin method)

Q: Could we use this technique for a differential equation with 100 dimensions?

- ▶ **A.** No chance!
- ▶ If we have N grid points in very dimension, then total matrix dimension is N^{100} .
- ▶ Need to go to deep learning to solve high dimensional problems.